

Nutkin

January 28, 2026

1 Overview

The **Nutkin** software implements a linear-based variability metric and a corresponding differential testing framework for quantifying and testing differences in overall transcriptional cell-to-cell variability between groups of cells using single-cell RNA-seq data. We expect Nutkin to help more researchers gain a deeper understanding of cellular heterogeneity in various biological and biomedical contexts by providing interpretable statistical insights into overall transcriptional variations.

In this manual, we describe how to preprocess single-cell RNA-seq data and produce the inputs required by **Nutkin** in Section 2, and how to perform an analysis using **Nutkin**, including descriptions of input file formats and output files, in Section 3. As example data, we use scRNA-seq data from GEO (GSM7872694), which corresponds to 10X Genomics 3' scRNA v3 of SPLINTR-barcoded MLL-AF9 cells cultured in vitro. The processed barcode data (`test_data.h5ad`) are provided in the `../Nutkin/data/` directory, but in Section 5 we also provide instructions on how to generate `test_data.h5ad` from the raw data files downloaded from <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSM7872694>. Users can either use the provided `test_data.h5ad` file or produce their own `test_data.h5ad` from the raw data following the same steps to preprocess and perform Nutkin analysis as described in Sections 2 and 3.

2 Single-cell RNA-seq data preprocessing

The **Nutkin** software takes single-cell RNA-seq data in the form of an `AnnData` object (`.h5ad` format) as input. When an `AnnData` object is provided, Nutkin automatically detects whether preprocessing has already been performed. If not, preprocessing and normalization are conducted using Scanpy in `Nutkin/nutkin/preprocessing.py`, producing the inputs required for **Nutkin**. If the input is not an `AnnData` object, an error is raised. Here, we describe how to use the

main function `load_data`, using barcode data as an example. The file `Nutkin/test/test.ipynb` contains the Python scripts used in this section.

The main function `load_data` provides a complete pipeline for handling single-cell `.h5ad` datasets. It starts by loading the input file and automatically detects whether the data has already been preprocessed by checking for `log1p` in `adata.uns` or `highly_variable` in `adata.var.columns`. If the data is raw, the function executes a core preprocessing routine that includes filtering low-quality cells and genes, normalizing total counts, log-transforming, selecting highly variable genes, and scaling the data. After preprocessing, `load_data` can optionally perform PCA, storing the results in the `AnnData` object and generating a scree plot saved to a user-specified output directory. Users can also generate low-dimensional embeddings (UMAP), computed from the PCA representation and colored according to a specified group column; the embedding plots are likewise saved if an output path is provided. The function returns the processed `AnnData` object containing all computed PCA and embedding results, which is ready for downstream analyses.

Open a Python session and read the functions in `nutkin/preprocessing.py`:

```
import sys
import os
import scanpy as sc

# Append the parent directory of Nutkin to the Python path
project_root = os.path.abspath("..")
sys.path.append(project_root)

# Import the preprocessing module
import nutkin.preprocessing
```

Then, set a path to the directory containing the barcode data and read it using `Scanpy`:

```
data_path = "../Nutkin/data/test_data.h5ad"
adata = sc.read_h5ad(data_path)
print(adata)

# AnnData object with n_obs × n_vars = 1932 × 31053
#   obs: 'barcode', 'bc.umi.count', 'num_barcodes', 'max_umi',
#         'lineage_barcodes_filtered', 'umi_counts_filtered',
#         'clone_id', 'louvain', 'clone_plot_group'
#   var: 'gene_ids'
#   uns: 'log1p', 'louvain', 'louvain_colors', 'neighbors', 'pca', 'umap'
```

```
# obsm: 'X_pca', 'X_umap'
# varm: 'PCs'
# obsp: 'connectivities', 'distances'
```

We set a path to the output directory and run `load_data`. Here, we use default values for `n_comps` and `run_pca`, which correspond to 50 principal components to compute during PCA and generate the scree plot. We also set `run_embedding=True` to generate a low-dimensional embedding (UMAP) and use `clone_id` in `adata.obs` to color the plot and group.

```
output_path = "../Nutkin/test/result/"
adata = nutkin.preprocessing.load_data(
    data_path=data_path,
    output_path=output_path,
    n_comps=50,
    run_pca=True,
    run_embedding=True,
    group_col='clone_id'
)

# Output:
# Successfully loaded data from: ../Nutkin/data/test_data.h5ad
# Data detected as PREPROCESSED. Skipping core preprocessing steps.
# Generating UMAP embedding...
# WARNING: saving figure to file ../Nutkin/test/result/umapumap_plot.png
```

Finally, we inspect the output files in `../Nutkin/test/result/`, which include `preprocessed_data.h5ad`, `pca_scree_plot.png`, and `umapumap_plot.png`.

3 Nutkin

3.1 Input file format for Nutkin

Nutkin requires single-cell RNA-seq data in `.h5ad` format, as produced by `nutkin/preprocessing.py`.

3.2 Performing an analysis using Nutkin

If you followed the procedure in Section 2, the `preprocessed_data.h5ad` file can be found in the `../Nutkin/test/result/` directory. We then load this preprocessed dataset and create an instance of the Nutkin class from `Nutkin/nutkin/nutkin.py`. This step initializes the Nutkin

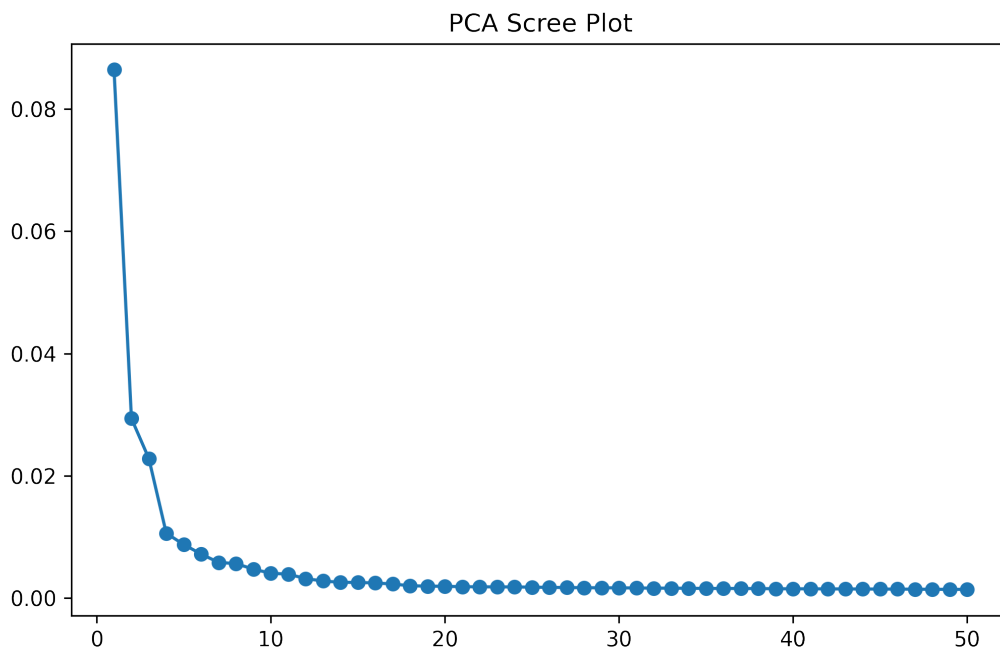


Figure 1: PCA scree plot

workflow by associating the AnnData object with the specified group structure and setting up all metric-related components required for downstream analyses.

The parameters used in the `Nutkin` class are defined as follows. The input `adata` is an AnnData object containing preprocessed single-cell RNA-seq data. The parameter `num_pc` specifies the number of principal components retained for downstream PCA-based analysis and defined in `nutkin/config.py`. Group membership is determined by `group_col`, which indicates the column in `adata.obs` storing group labels, while `group1` and `group2` define the two sets of groups to be compared and may consist of single or multiple group names. The parameter `metric_type` controls how variability is summarized across principal components, allowing aggregation by sum, product, or both. Variability within each group is quantified according to `variability_method`, which supports standard deviation, median absolute deviation or both. The logical flag `verbal` determines whether progress information is printed during execution, whereas `detail` controls the generation of detailed intermediate outputs and diagnostic visualizations. Finally, `plot_pc_num` specifies the number of principal components used for visualization, and `plot_type` determines the form of PCA-based plots, including scatter, contour, or both.

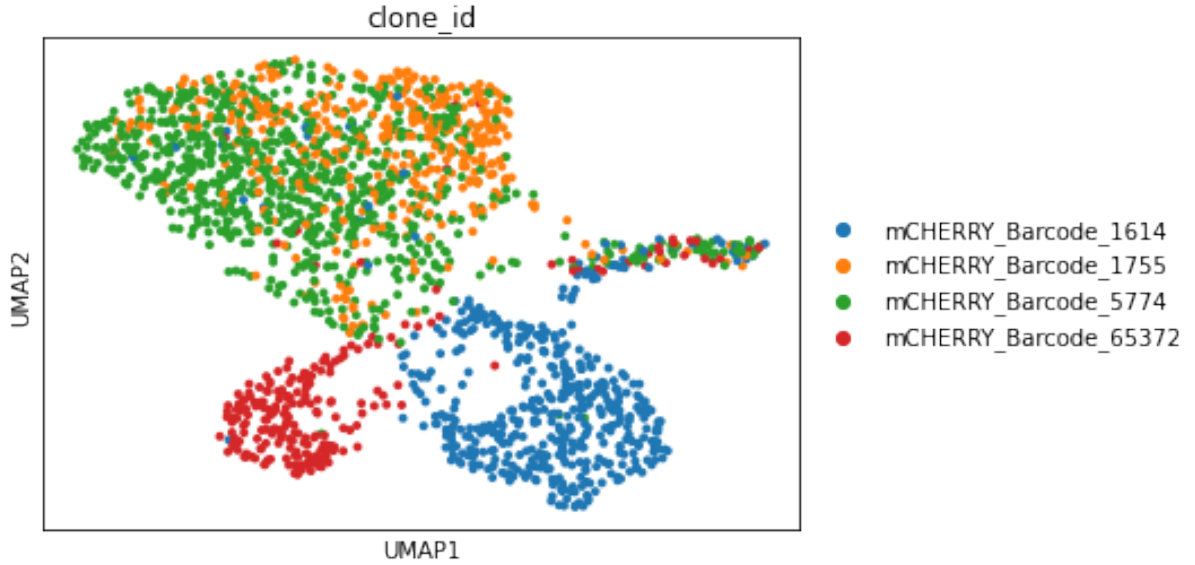


Figure 2: UMAP plot

For the test example, `Nutkin` is initialized using the preprocessed `AnnData` object with most parameters left at their default settings, namely `num_pc` set to the number of principal components defined in `nutkin/config.py`, `metric_type` set to "sum", `variability_method` set to "mad", `verbal` set to `True`, and `plot_pc_num` set to 3. In this example, `group_col` is explicitly set to "clone_id" to define group labels, `detail=True` is used to enable detailed PCA-related outputs and visualizations, and `plot_type` is set to "both".

```
adata = sc.read("../test/result/preprocessed_data.h5ad")
from nutkin.nutkin import Nutkin
nut = Nutkin(
    adata=adata,
    group_col='clone_id',
    output_path=output_path,
    detail=True
)
```

3.2.1 Quantify the variability for each group

First, we use `Nutkin` to quantify the variability for each cell group in the dataset. The specific computation of variability follows the procedure described in our paper. The resulting variability measurements are saved as `var_measurement.csv` in the output directory `../Nutkin/test/result/`,

including for each group the group name, the number of cells, and the computed variability.

```
nut.measure()
# Quantifying variability using MAD method
# Running dimensional reduction (PCA)...
# Saving results
```

	group	number of cells	sum statistics
0	mCHERRY_Barcode_1614	492	2.885429
1	mCHERRY_Barcode_65372	229	2.891356
2	mCHERRY_Barcode_1755	404	2.942356
3	mCHERRY_Barcode_5774	807	3.026228

3.2.2 Compare variability between selected groups

After computing and saving the variability measurements, we perform differential testing between selected cell groups using Nutkin. The null hypothesis assumes no difference in variability between the two groups. A two-sided p-value less than 0.05 is considered statistically significant, in which case the null hypothesis is rejected, indicating that the variability of the two groups differ significantly.

In this test case, we first assign `group1` and `group2` to `['mCHERRY_Barcode_1614', 'mCHERRY_Barcode_1614']` and `['mCHERRY_Barcode_5774', 'mCHERRY_Barcode_1755']`, respectively, so that these two sets of groups are compared directly.

```
nut.differential_test(group1=['mCHERRY_Barcode_1614', 'mCHERRY_Barcode_1614'],
                    group2=['mCHERRY_Barcode_5774', 'mCHERRY_Barcode_1755'])
# Conducting differential analysis using MAD method
# ['mCHERRY_Barcode_1614', 'mCHERRY_Barcode_1614']
# ['mCHERRY_Barcode_5774', 'mCHERRY_Barcode_1755']
# Comparing mCHERRY_Barcode_1614 and mCHERRY_Barcode_5774
# Running Bootstrapping
# Comparing mCHERRY_Barcode_1614 and mCHERRY_Barcode_1755
# Running Bootstrapping
# Saving results
```

Subsequently, we run the differential test with both `group1` and `group2` left empty. In this mode, Nutkin automatically performs pairwise comparisons between groups in the order they were quantified in the previous variability analysis.

	groupA	number of cells in groupA	sum statistics for groupA	groupB	number of cells in groupB	sum statistics for groupB	null hypothesis	test statistics sum	p value sum
0	mCHERRY_Barcode_1614	492	2.885429	mCHERRY_Barcode_5774	807	3.026228	A = B	-0.140799	0.060932
1	mCHERRY_Barcode_1614	492	2.885429	mCHERRY_Barcode_1755	404	2.942356	A = B	-0.056927	0.383612

```
nut.differential_test(group1=['mCHERRY_Barcode_1614', 'mCHERRY_Barcode_1614'],
                    group2=['mCHERRY_Barcode_5774', 'mCHERRY_Barcode_1755'])
# Conducting differential analysis using MAD method
# Comparing mCHERRY_Barcode_1614 and mCHERRY_Barcode_65372
# Running Bootstrapping
# Comparing mCHERRY_Barcode_65372 and mCHERRY_Barcode_1755
# Running Bootstrapping
# Comparing mCHERRY_Barcode_1755 and mCHERRY_Barcode_5774
# Running Bootstrapping
# Saving results
```

	groupA	number of cells in groupA	sum statistics for groupA	groupB	number of cells in groupB	sum statistics for groupB	null hypothesis	test statistics sum	p value sum
0	mCHERRY_Barcode_1614	492	2.885429	mCHERRY_Barcode_65372	229	2.891356	A = B	-0.005927	0.976553
1	mCHERRY_Barcode_65372	229	2.891356	mCHERRY_Barcode_1755	404	2.942356	A = B	-0.051001	0.392156
2	mCHERRY_Barcode_1755	404	2.942356	mCHERRY_Barcode_5774	807	3.026228	A = B	-0.083871	0.284810

The output of the differential testing is saved as `differential_variability.csv` under `../Nutkin/test/result/`. Each row contains the comparison information, including `groupA` and its cell count, statistics for `groupA`, `groupB` and its cell count, statistics for `groupB`, the null hypothesis, the test statistic, and the associated two-sided p-value.

3.2.3 Visualize and summarize PCA results

Finally, we visualize and summarize the PCA results using function `visualize_pca_results` in `Nutkin`. In this test example, `detail=True` is set to generate detailed plots and summary tables, `plot_pc_num=3` specifies that the first three principal components are shown, and `plot_type="both"` enables both scatter and contour visualizations. The PCA plots are generated for each pair of cell groups, with the group pairings corresponding to those used in the previous differential variability comparisons. This step produces graphical representations of the PCA structure as well as tabular outputs summarizing the variability of each principal component for every cell group.

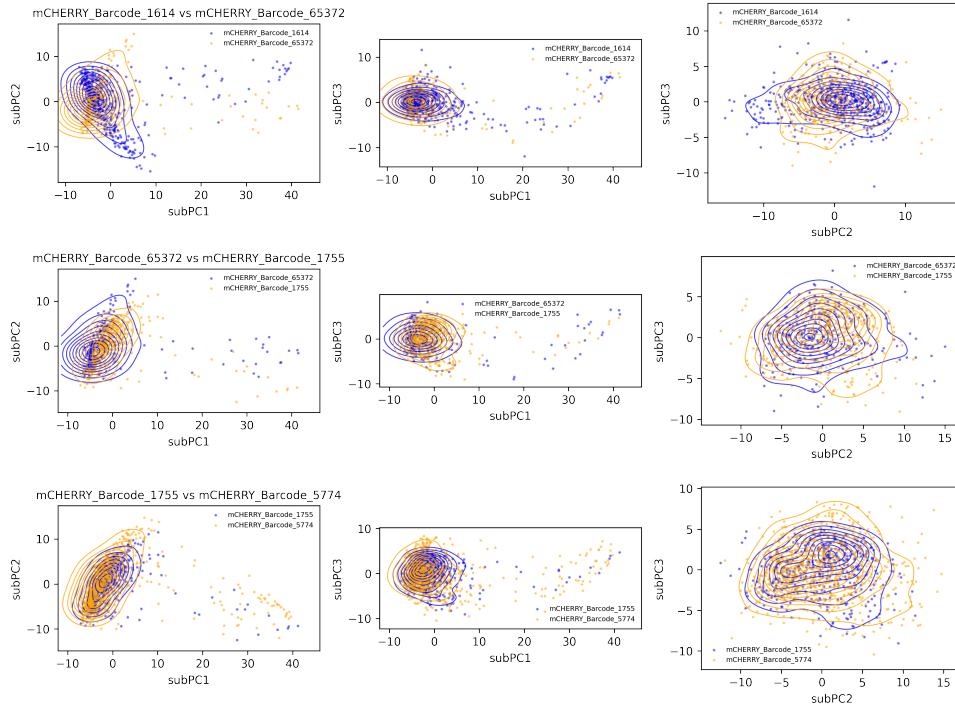
```

nut.visualize_pca_results(detail=True, plot_type='both')
# [('mCHERRY_Barcode_1614', 'mCHERRY_Barcode_65372'),
   ('mCHERRY_Barcode_65372', 'mCHERRY_Barcode_1755'),
   ('mCHERRY_Barcode_1755', 'mCHERRY_Barcode_5774')]
# PCA visualization (both) saved to ../Nutkin/test/result

```

The resulting figure (`pairwise_pca_both.png`) and summary table (`summary_specific.csv`) are saved in the output directory `../Nutkin/test/result`. The summary table includes the group names and the variability measurements for the first fifteen principal components (PC1-PC15) as well as the overall sum. The figure shows scatter plots and contour plots for each pair of groups and each pair of principal components, providing a visual representation of the PCA structure and group-level variability.

Groups	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	PC11	PC12	PC13	PC14	PC15	sum
mCHERRY_Barcode_5774	8.7691147	4.7730664	3.8901170	2.8731296	2.5011250	2.2841527	2.1603555	1.9281584	1.8826837	1.7163640	1.5796511	1.5156076	1.4157712	1.0283640	0.7949366	3.0262275
mCHERRY_Barcode_1614	8.6112611	4.9992339	3.3316689	2.4711124	2.1914871	1.9891092	1.9101232	1.8033577	1.6771136	1.5207630	1.3313579	1.1183446	1.0624052	0.8154502	0.6742550	2.885429
mCHERRY_Barcode_65372	10.303333	4.444994	3.6253988	2.6904302	2.2825488	2.1533302	2.0048133	1.8865808	1.7056856	1.6179419	1.4489939	1.4292383	1.0998608	1.0017128	0.6234535	2.8913555
mCHERRY_Barcode_1755	6.6994285	4.4990037	3.7166152	2.5807445	2.2444401	2.0858395	2.0711128	1.8249142	1.7229046	1.6191822	1.3843939	1.2528330	1.1537458	0.7960366	0.7106716	2.942356



4 Run Nutkin from the command line

Nutkin can also be run directly from the command line using the same parameters as in the previous example. First, navigate to the `nutkin` directory:

```
cd ../nutkin
```

Then execute the main script with the desired input and output paths:

```
python -m nutkin.main \  
  --input_path ../Nutkin/data/test_data.h5ad \  
  --output_path ../Nutkin/test/result/ \  
  --group_col clone_id \  
  --group1 mCHERRY_Barcode_1614 mCHERRY_Barcode_1614 \  
  --group2 mCHERRY_Barcode_5774 mCHERRY_Barcode_1755 \  
  --detail \  
  --plot_type both
```

The outputs produced by this command are identical to those obtained using the Python API example described in the previous sections.

5 Preparing scRNA-seq data for Nutkin analysis

The test dataset `test_data.h5ad` was generated from the raw scRNA-seq data originally obtained from GEO (GSM7872694), which includes the following files:

- `GSM7872694_BARTab_cell-barcode-anno.tsv.gz` (103.6 Kb)
- `GSM7872694_barcodes.tsv.gz` (68.5 Kb)
- `GSM7872694_features.tsv.gz` (245.0 Kb)
- `GSM7872694_matrix.mtx.gz` (145.7 Mb)

The processing steps to create `test_data.h5ad` were as follows. First, the raw count matrix in MTX format was read and transposed to ensure cells correspond to rows and genes to columns. The gene names and cell barcodes were assigned using the accompanying feature and barcode files. Lineage barcode annotations from `GSM7872694_BARTab_cell-barcode-anno.tsv.gz` were parsed to compute the number of barcodes detected per cell and the maximum UMI count per barcode. Minor barcodes supported by fewer than half of the maximum UMI in each cell were filtered. Finally, the processed count matrix and the filtered BARTab annotations were

merged into a `scanpy AnnData` object. The resulting `AnnData` object contains the count matrix, gene metadata (`gene_ids` and gene symbols), and cell-level `BARTab` annotations aligned to `adata.obs`.

To reduce the dataset size for this test example, only cells corresponding to the following four clones were selected: `mCHERRY_Barcode_1614`, `mCHERRY_Barcode_5774`, `mCHERRY_Barcode_1755`, and `mCHERRY_Barcode_65372`. The filtered `AnnData` object, containing only these cells along with their gene expression counts and `BARTab` annotations, was saved as `test_data.h5ad`. This smaller dataset serves as the input for Nutkin analysis in the following sections.